

Course Introduction
Video • 1 min

Welcome to the Course
Reading • 10 min

Course Feedback
Video • 1 min

Pre-Course Survey
Reading • 1 min

Pre-Course Survey
Ungraded Plugin • 25 min

Week 1 Introduction
Video • 1 min

Introduction
Reading • 15 min

Customer #1: Use Case and Requirements
Video • 5 min

Customer #1: Requirements Breakdown
Video • 5 min

Selecting a Serverless Compute Service
Video • 9 min

AWS Lambda Exploration
Video • 1 min

Compute on AWS
Reading • 15 min

Choosing an AWS Database Service
Video • 10 min

DynamoDB Exploration
Video • 1 min

Databases on AWS
Reading • 25 min

Building Event-Driven Architectures
Video • 5 min

SNS Exploration
Video • 3 min

Event-Driven Architectures on AWS
Reading • 25 min

Decoupling AWS Solutions
Video • 1 min

SQS Exploration
Video • 5 min

Decoupling Solutions on AWS
Reading • 25 min

Customer-Centric Serverless Architecture Design
Role Play • 15 min

Customer #1: Solution Overview
Video • 4 min

Week Wrap-up: Taking This Architecture to the Next Level
Video • 5 min

Architecture Optimizations for Week 1
Reading • 15 min

Exercise 1: Walkthrough
Video • 20 min

Challenge: Building a Proof of Concept
Reading • 10 min

Week 1 Assessment
Graded Assignment • 25 min

Mid-Course Survey
Reading • 1 min

Mid-Course Survey
Ungraded Plugin • 25 min

Module 2
Designing a serverless data analytics solution on AWS

Module 3
Designing a hybrid solution for container-based workloads on AWS

Module 4
Designing a solution following account governance and management best practices

Event-Driven Architectures on AWS

This week's customer currently uses a synchronous web application to host the orders service, which is causing various issues—for example, the code is too tightly coupled with downstream API calls. Morgan suggested that they move to an event-driven architecture to solve this problem.

An event-driven architecture uses events to invoke and communicate between decoupled services. It's a common architecture in modern applications that are built with microservices. An event is a change in state, or an update, such as placing an item in a shopping cart on an ecommerce website. Events can either carry the state (the item purchased, its price, and a delivery address) or events can be identifiers (a notification that an order was shipped).

Event-driven architectures have three key components: event producers, event routers, and event consumers. A producer publishes an event to the router, which filters and pushes the events to consumers. Producer services and consumer services are decoupled, which means that they can be scaled, updated, and deployed independently.

The following diagram shows an example of an event-driven architecture for an ecommerce site. By using this architecture, the site can react to changes from various sources during times of peak demand, without crashing the application or overprovisioning resources.

For more information about event-driven architectures, see [What is an Event-Driven Architecture?](#)

Amazon EventBridge compared to Amazon SNS

You can use both Amazon EventBridge and Amazon Simple Notification Service (Amazon SNS) to develop event-driven applications. Your choice will depend on your specific needs.

We recommend EventBridge when you want to build an application that reacts to events from software as a service (SaaS) applications or AWS services. EventBridge is the only event-based service that integrates directly with third-party SaaS AWS Partners. EventBridge also automatically ingests events from over 100 AWS services without requiring developers to create any resources in their account. In addition, EventBridge uses a defined, JSON-based structure for events, and you can also select events to forward to a target by creating rules that are applied across the entire event body. EventBridge currently supports over 15 AWS services as targets, including AWS Lambda, Amazon Simple Queue Service (Amazon SQS), Amazon SNS, Amazon Kinesis Data Streams, and Amazon Kinesis Data Firehose, and more. At launch, EventBridge has limited throughput (see the service limits), which can be increased on request. It also has a typical latency of about half a second.

We recommend Amazon SNS when you want to build an application that reacts to high throughput or low-latency messages that are published by other applications or microservices. Amazon SNS provides nearly unlimited throughput. You can also use it for applications that need very high fan-out (thousands or millions of endpoints). Messages are unstructured and can be in any format. Amazon SNS supports forwarding messages to six different types of targets, including AWS Lambda, Amazon SQS, HTTP endpoints, Short Message Service (SMS), mobile push, and email. The typical latency of Amazon SNS typical is under 30 milliseconds. A range of AWS services—more than 30, including Amazon Elastic Compute Cloud (Amazon EC2), Amazon Simple Storage Service (Amazon S3), and Amazon Relational Database Service (Amazon RDS)—send SNS messages by configuring the service to send them.

Amazon EventBridge

EventBridge is a serverless event bus service that you can use to connect your applications with data from various sources. EventBridge delivers a stream of real-time data from your applications, software as a service (SaaS) applications, and AWS services to targets such as AWS Lambda functions, HTTP invocation endpoints using API destinations, or event buses in other AWS accounts.

EventBridge receives an event, which is an indicator of a change in environment. EventBridge then applies a rule to route the event to a [target](#). Rules match events to targets based on either the structure of the event (which is called an event pattern), or on a schedule. For example, when an EC2 instance changes from pending to running, you can have a rule that sends the event to a Lambda function. For more information about events, see [Amazon EventBridge events](#). For more information about rules, see [Amazon EventBridge rules](#). Finally, for more information about event patterns, see [Amazon EventBridge event patterns](#).

All events that come to EventBridge are associated with an event bus. Rules are tied to a single event bus, so they can only be applied to events on that event bus. Your account has a default event bus, which receives events from AWS services. You can also create custom event buses to send or receive events from a different account or Region. For more information about event buses, see [Amazon EventBridge event buses](#).

For more information about Amazon EventBridge, see the [Amazon EventBridge FAQs](#) or [What is Amazon EventBridge?](#)

Amazon SNS

Amazon SNS is a managed service that provides message delivery from publishers (which are also known as producers and consumers). Publishers communicate asynchronously with subscribers by sending messages to a topic, which is a logical access point and communication channel. Clients can subscribe to the SNS topic and receive published messages by using a supported endpoint type, such as Amazon Kinesis Data Firehose, Amazon SQS, AWS Lambda, HTTP, email, mobile push notifications, and mobile text messages through Short Message Service (SMS).

Morgan chose Amazon SNS the customer's architecture because it's simple to use and supports a straightforward way to send messages between application components.

Service Update: Amazon SNS now supports payload-based message filtering [click here](#) for more information.

- For more information about Amazon SNS, see the [Amazon SNS FAQs](#) or [What is Amazon SNS?](#)
- For a tutorial on how to set up an Amazon SNS topic, see [Get started with Amazon SNS](#).

Amazon DynamoDB Streams

DynamoDB Streams captures a time-ordered sequence of item-level modifications in any DynamoDB table, and stores this information in a log for up to 24 hours. Applications can access this log and view the data items as they appeared, before and after they were modified, in near real time. Encryption at rest encrypts the data in DynamoDB streams. DynamoDB Streams helps ensure the following:

- Each stream record appears exactly one time in the stream.
- For each item that is modified in a DynamoDB table, the stream records appear in the same sequence as the actual modifications to the item.

DynamoDB Streams writes stream records in near real time so that you can build applications that consume these streams and take action based on the contents. You can enable a stream on a new table when you create it by using the AWS Command Line Interface (AWS CLI) or one of the AWS SDKs. You can also enable or disable a stream on an existing table, or change the settings of a stream. DynamoDB Streams operates asynchronously, so table performance isn't affected if you enable a stream. All data in DynamoDB Streams is subject to a 24-hour lifetime. You can retrieve and analyze the last 24 hours of activity for any given table. However, data that is older than 24 hours is susceptible to trimming (removal) at any moment.

- For more information about Amazon DynamoDB Streams, see [Change data capture for DynamoDB Streams](#).
- For a tutorial about how to process DynamoDB streams with an AWS Lambda function, see [Tutorial: Process new items with DynamoDB Streams and Lambda](#).

Mark as completed

Like

Disk

Report an issue

coach

Hi, Venkatesh!

How can I help?

Explain this topic in simple terms

Give me a summary

Give me practice questions

Give me real-life examples

Ask me anything

Go to next item

Coach is powered by AI, so check for mistakes and don't share sensitive info. Your data will be used in accordance with [Data Privacy Notice](#)